B.7 Lab 7: Computing Eigenvalues

Problem 1: The Deflation Method

Recall that the power method is an iterative approach for computing the eigenvector corresponding to the dominant eigenvalue of an $n \times n$ matrix A. Once the dominant eigenpair, $(\lambda_1, \mathbf{x}_1)$ is computed, we may wish to compute the next most dominant eigenvalue, λ_2, where

$$|\lambda_1| \geq |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|.$$

For a symmetric matrix A, one can show that if $\mathbf{y}_1 = \frac{\mathbf{x}_1}{\|\mathbf{x}_1\|}$, then the matrix

$$\tilde{A} = A - \lambda_1 \mathbf{y}_1 \mathbf{y}_1^T$$

has eigenvalues

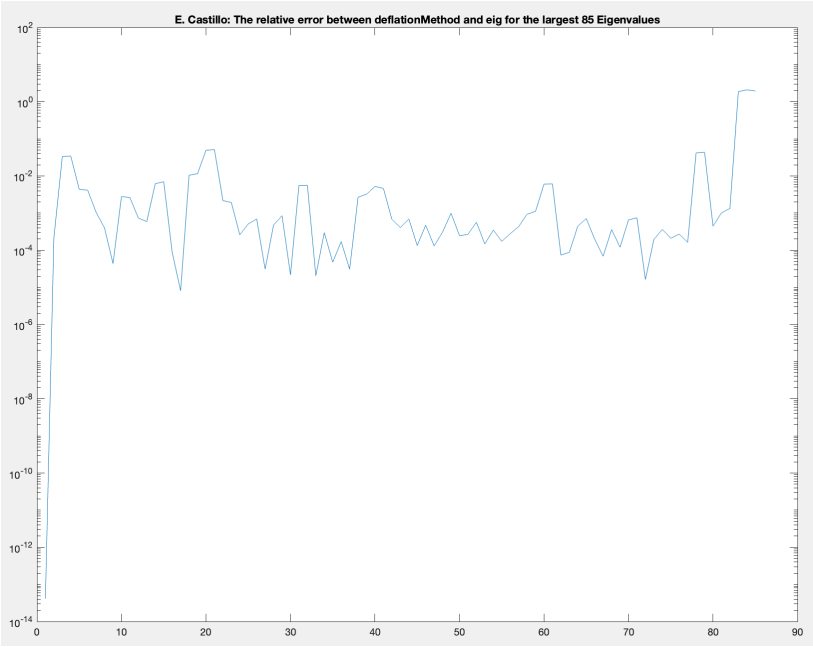
$$0, \lambda_2, \lambda_3, \ldots, \lambda_n$$

and the eigenvectors of $\tilde{A}$ are eigenvectors of A. Therefore, we can find $(\lambda_2, \mathbf{x}_2)$ by applying the power method to $\tilde{A}$. This strategy is known as the ***deflation method***. The deflation method can be applied iteratively to find multiple eigenpairs. Note that each application of the power method requires its target matrix to have a dominant eigenvalue, so computing the k largest magnitude eigenvalues this way implicitly assumes strict inequalities between them:

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_k|.$$

In addition, in general, the deflation method becomes more inaccurate as we calculate more eigenvalues because the approximation errors introduced at each iteration accumulate as the process continues.

- Write a function `deflationMethod.m` that computes the k largest magnitude eigenvalues of an $n \times n$ symmetric matrix. You will need a power method subfunction. The power method is already implemented in Listing 6.1 of Chapter 6, so include it as the subfunction, called with a tolerance that solves each eigenvalue estimate to 6 significant digits.
- Download `testMat.mat`. It contains a symmetric matrix A. Write a function `deflationTest.m` that calls your `deflationMethod.m` to find the 85 largest magnitude eigenvalues for the matrix A. Call MATLAB's `eig` function to compute all of A's eigenvalues. Then, create a `semilogy` plot of the relative error between each of your 85 estimates (in descending order) and the corresponding eigenvalues computed by the `eig` function. It should look something like this:



Problem 2: Predicting the Cost of an Iterative Solve

In Lab 6, Section B.6, you measured the number of iterations the Jacobi, Gauss-Seidel, and SOR methods require for each additional digit of accuracy when solving the steady state heat equation system, and the cost per digit came out to a fixed number for each method. Section 6.11 supplies the explanation: each iteration shrinks the slowest-dying error component by the spectral radius $\rho(G)$ of the iteration matrix $G = M^{-1}N$, where $N = M - A$.

Write a script `spectralRadiusStudy.m` that constructs the heat equation system `[A,b,D] = laplacianMat([0 1 0 1])`, forms the iteration matrix for each of the three methods using the splittings of Chapter 5 ($M = D$ for Jacobi, $M = D + L$ for Gauss-Seidel, and $M = \frac{1}{\lambda}D + L$ with $\lambda = 1.75$ for SOR), and computes the spectral radius of each iteration matrix using MATLAB's `eig`. Starting from the fact that each iteration multiplies the error by the factor $\rho(G)$, derive a formula for the number of iterations required per digit of accuracy, and state the derivation in your header. Your script should display, for each method, the spectral radius, the predicted iterations per digit, and the cost per digit you measured in Lab 6.

Finally, compare the Gauss-Seidel spectral radius against the square of the Jacobi spectral radius. In your header, explain how this comparison accounts for the observation, recorded in Chapter 5, that Gauss-Seidel cuts the Jacobi iteration count roughly in half. One caveat: the exact relationship you will find is a theorem for the special family of matrices that finite difference discretizations like this one produce [26], not a property of matrices in general, and its proof belongs to a more advanced course.

General Submission Guidelines

```
[FirstName][LastName]_[filename].m            For programs
[FirstName][LastName]_[filename]_plot_1.pdf   For corresponding plots (export them to pdf)
[FirstName][LastName]_[filename]_plot_2.pdf   For problem with more than one plot
```

Submission Checklist

- Problem 1: `deflationMethod.m`
  - Fully commented code and complete header
  - Power method subfunction with the correct tolerance
  - Correct implementation of the deflation method iteration
- `deflationTest.m`
  - Correct code and error plot
- Problem 2: `spectralRadiusStudy.m`
  - Complete header containing the iterations-per-digit derivation
  - Correct construction of the three iteration matrices
  - Correct spectral radii with the predicted versus measured comparison displayed
  - Header explanation of the Gauss-Seidel versus squared Jacobi comparison